

Как разогнать PHP-движок слотов с 5 000 до 300 000+ спинов в секунду и сэкономить миллионы на инфраструктуре

Автор: Andrii Starkov (Senior/Staff Software Engineer)

Репозиторий проекта: [GitHub Repository](#)

Профессиональные контакты: [LinkedIn Profile](#)

Содержание

1. **Введение** (Стенд, базовая игра, ссылки на GitHub/LinkedIn)
2. **Анализ технического долга в iGaming** (Специфика расчетов, легасы, Монте-Карло как Bottleneck)
3. **Архитектурные решения для Ultra Performance на PHP** (Убийство ООП, плоские массивы, Short-Circuit, микро-оптимизации)
4. **Результаты бенчмарков** (Сравнение скорости 5k против 304k спинов/сек, профит для бизнеса)
5. **Граница разумного: Темная магия оптимизации** (Битмаски, процедурный хаос, предвычисленные карты комбинаций)
6. **Миграция на Go (Golang) за 3 недели** (Строгая типизация, оптимизация слайсов, миллионы RPS)
7. **Заключение** (Мифы о PHP, «механическое сочувствие» к железу)

1. Введение

Привет! Я Senior/Staff Software Engineer с более чем 11-летним опытом в backend/frontend-разработке, проектировании систем и enterprise-архитектуре. Мой профиль — высоконагруженные системы с тяжелым legacy, сложной бизнес-логикой и рефакторинг монолитов с десятилетней историей «на лету», без остановки production.

Эта статья — результат глубокого анализа архитектурных проблем на сотнях реальных игр и кодовой базе, которая копилась более десяти лет. Я полностью переписал игровое ядро с нуля, вынеся всю логику взаимодействия с платформой за его пределы. Движок стал чистой математикой.

Ниже — подробный технический гайд о том, как заставить PHP выдавать производительность коммерческих вычислительных ядер, почему «красивый» код убивает процессор, и как инженерные решения могут сэкономить бизнесу миллионы долларов.

Окружение и характеристики испытательного стенда

Все бенчмарки и показатели производительности, о которых пойдет речь, зафиксированы на одном и том же оборудовании:

- **ОС:** Debian GNU/Linux 12 (bookworm) — Ядро Linux 6.1.0-44-amd64 (x86_64)
- **СРУ:** 12th Gen Intel(R) Core(TM) i7-1260P (12 ядер / 16 потоков, до 4.70 GHz)

- **RAM:** 32 GB
- **NVME:** Samsung SSD 990 PRO 1TB
- **Базовая игра:** Для чистоты эксперимента взята простая модель — игровое поле 3x3, 7 символов, 3 выигрышные линии (горизонталь слева направо). Никаких фриспинов, респинов, бонусных раундов и дополнительных механик.

2. Анализ технического долга в iGaming

Давайте начистоту: конечному клиенту абсолютно плевать, насколько красивый код написан у вас в репозитории. Игра крутится, слоты мигают, баланс сходится — для него система работает. На коленке можно писать и 20 лет, но для бизнеса внутренняя архитектура — это либо ракетное топливо, либо бетонная плита на ногах.

Мне пришлось столкнуться с кодовой базой крупного iGaming-проекта со шлейфом из сотен игр, которая представляла собой хрестоматийный музей технического долга. Вот лишь малая часть:

- **Математика во float:** Все расчеты велись в плавающей точке, генерируя классические проблемы округлений и расчетов.
- **Версионирование через «простыни» switch-case:** Использовалось для поддержки старых игр.
- **GodRepository:** Все игры и инструменты находились в едином месте.
- **Файлы-монстры:** Один файл мог раздуваться до десятков тысяч строк кода, где сосуществовали по 10 функций с диким неймингом, которые делали абсолютно одно и то же, но «каждая по-своему».
- **Вакуум вместо тестов и документации:** Кодовая база передавалась как легенды и мифы древней Греции — из уст в уста.

Когда у компании в портфеле сотни игр, а на платформе одновременно крутят барабаны десятки тысяч игроков, нагрузка на сервера становится колоссальной. Это не классический веб-CRUD. В стандартном вебе процессор в основном простаивает, ожидая ответа от базы данных или сети. Если там нужно убрать оверхед на инициализацию PHP-приложения на каждый запрос, мы просто берем что-то вроде RoadRunner или Swoole, которые держат воркеры в памяти.

Но в iGaming сервер непрерывно занят чистыми, тяжелыми вычислениями игровой логики на каждый чих игрока. И вся эта история сильно греет CPU, особенно когда нужно провести Монте-Карло эмуляции — обчислить 10 000 000 000 (10 миллиардов) спинов или других взаимодействий игрока с игрой для каждого RTP профиля. Это обязательный этап для валидации математики, расчета волатильности (Coefficient of Variation) и верификации RTP перед отправкой софта в жесткие сертификационные лаборатории (iTechLabs, eCOGRA).

Если писать игровой PHP движок по академическим канонам «чистого ООП» (с объектами под каждый символ, линию и результат спина), PHP захлебнулся бы на аллокациях памяти и выдал жалкие 100–200 спинов в секунду. Проверено на опыте.

Старый легаси-код писался во времена динозавров. Он забит тяжелыми системными функциями (к примеру, `array_merge`, `array_flip`) и другими избыточными проверками, глубокими вложенными структурами (вызывающими Cache Miss) и постоянным взаимодействием с файловой системой для

сессий и логирования. Его средняя производительность на эмуляции составляла примерно ~5 000 итераций (спинов, покупки и т.д.) в секунду.

Сухая математика бизнес-потерь

Если вам нужно прогнать базовые 10 миллиардов итераций при скорости старого легаси-движка в 5 000 спинов/сек, мы получаем катастрофический Bottleneck:

$$10,000,000,000 / (5,000 \times 3,600) \approx 555.6 \text{ часов на одном ядре}$$

- На 10 ядрах параллельно — это 55.5 часов непрерывной работы.
- На 20 ядрах — 27.7 часов.

И это только один прогон. А таких циклов требуются десятки или сотни на одну игру. Ведь малейший сдвиг вероятности выпадения символа или кастомной фишки влияют не только на отдачу но и на retention, делая игру либо разорительной для оператора, либо скучной для игрока.

В итоге показатель Time-to-Market (TTM) растягивается на недели, разработчики и математики днями ждут результатов симуляций, а компания жжет колоссальные бюджеты на аренду многоядерных серверов в облаках.

Новое ядро изначально проектировалось так, чтобы исправить обе проблемы: сделать код читаемым, масштабируемым (где сам код и есть документация) и при этом выжать максимум производительности из железа.

Ниже представлен детальный разбор того, как снять оковы интерпретатора, избавиться от мусорных операций с памятью и заставить PHP 8.5 JIT работать на пределе физических возможностей процессора, выдавая 300 000+ итераций взаимодействия игрока с игрой (e.g. spin/buy) в секунду.

3. Архитектурные решения для Ultra Performance на PHP

Уничтожение памяти: Убийство ООП в Hot Path

Главный враг экстремальной производительности в PHP — это выделение памяти (Memory Allocation) и сборщик мусора (Garbage Collection). Циклы генерации игровых полей, поиска линий и расчета выплат — это наш Hot Path (горячий участок кода), который во время симуляций выполняется миллиарды раз.

В архитектуре есть код, который делегирует, направляет и структурирует — там паттерны программирования (порождающие, структурные, поведенческие) жизненно необходимы. Но внутри Hot Path вся эта академическая красота должна умереть. Здесь код обязан быть максимально тупым, прямолинейным и примитивным, словно написанным в каменном веке.

Объекты идут под нож

Никаких объектов внутри цикла. Создание даже самого легковесного объекта вроде `new WinLine()` или `new SpinResult()` внутри Hot Path заставляет процессор тратить сотни тактов на аллокацию структуры в памяти, а затем столько же — на её последующий обход и очистку сборщиком мусора. В горячем цикле имеют право существовать только примитивные типы данных и плоские массивы.

Осознанный компромисс: Да, массивы в PHP — это тоже структуры данных, и на них тоже выделяется память. Да, отказ от объектов в пользу плоских массивов превращает отладку и поддержку кода в головную боль. Но это фундаментальное ограничение самого языка, и мы сознательно идем на эту жертву. Это компромиссное решение: вне Hot Path мы по-прежнему можем писать красивый ООП-код, использовать объекты и абстракции, но перед самой отправкой данных в горячий цикл расчетов мы обязаны полностью «раздеть» их и конвертировать в плоские, простые примитивы.

Отказ от интерфейсов, динамики и переход на ручной SOLID

Никаких полиморфных вызовов, паттернов «Стратегия» или динамических переменных вида `$variable`. JIT-компилятор PHP 8.5 просто не способен оптимизировать код, если типы данных и пути выполнения постоянно меняются в рантайме. И процессору, и JIT-у нужен последовательный, абсолютно предсказуемый поток выполнения (flow control).

При этом мы не выбрасываем правила SOLID на помойку, но контроль за их соблюдением полностью переносится на разработчика — мы переходим на ручное управление архитектурой. Наша задача как инженеров — заранее заложить такую масштабируемость, чтобы под каждую новую игру не приходилось изобретать велосипед или переписывать систему с нуля. Чистота функций внутри Hot Path становится абсолютной: если метод `calculateWin()` спроектирован для расчета выигрышей, он занимается только расчетом выигрышей. В нем не может быть побочного логирования, и он не имеет права неявно модифицировать входные данные.

Запрет на клонирование

Забудьте про ключевое слово `clone`, и уж тем более про попытки глубокого клонирования через тяжелую сериализацию `serialize` / `unserialize`. Внутри горячего цикла это архитектурное преступление против процессора. Вся подготовка, копирование и трансформация данных должны завершиться до того, как сработает первая итерация.

Сплющивание матриц: Война с Cache Miss

Двумерные массивы — стандарт для отображения игровой сетки `Reels[X][Y]`. Но массив в PHP — это упорядоченная хэш-таблица (`zval`). Доступ к элементу `$reels[$x][$y]` заставляет процессор бесконечно прыгать по указателям. На больших объемах это приводит к L1/L2 Cache Miss (промахам кэша) — ядро процессора простаивает в ожидании данных из медленной RAM.

Решение: Склеить все барабаны слева направо, сверху вниз в один плоский одномерный массив. Игровая сетка 3x3 превращается в массив с индексами от 0 до 8.

- Было (2D): `$reels[reelId][rowId]`
- Стало (1D): `$reels[position]`

Для фиксированных полей координаты точки [X, Y] переводятся в плоский индекс по формуле:

$$position = rowID \times gameFieldWidth + reelID$$

Теперь выигрышная линия на самом верху сетки 3x3 — это просто проверка индексов 0, 1, 2. Данные лежат в памяти последовательно и идеально оседают в L1-кэше процессора.

Алгоритмическое пуленепробиваемое сокращение (Short-Circuit)

Когда речь заходит о миллиардах операций, даже малейшая оптимизация, экономящая всего 5% ресурсов — это потенциально сэкономленные десятки или сотни тысяч долларов. Оптимизация конкретных алгоритмов поиска выигрышей спокойно дает кратный xN буст скорости. Чтобы этого добиться, мы полностью отказываемся от универсальных калькуляторов. Вместо этого мы создаем отдельные изолированные методы или классы, заточенные под одну-единственную задачу: найти выигрыши одним конкретным способом.

В моем коде алгоритмы жестко разделены по классам. Они прогреваются ровно один раз на самом первом спине эмуляции и сохраняют в локальных параметрах объекта индексированный кэш и выделенную память, которая в дальнейшем просто обновляется. Таким образом, уже на втором спине калькулятор заранее знает высоту каждого барабана, длину поля и количество возможных выигрышей. Вся рабочая память под это дело уже выделена, закеширована и индексирована.

В качестве примера возьмем калькулятор линий. Вместо того чтобы в лоб перебирать все возможные комбинации на поле, мы применяем агрессивный ранний выход из циклов (Short-Circuit):

- **Проверка линий вместо комбинаций:** Если в игре всего 5 выигрышных линий, а потенциальных комбинаций символов — 25, мы пишем цикл строго по 5 линиям. На дистанции в 1 миллиард спинов это чистая экономия в 20 миллиардов итераций!
- **Вход со второго барабана:** Если по правилам игры комбинации считаются строго слева направо и должны содержать минимум 3 символа, то проверку линии можно начинать сразу со второго барабана (индекс 1). Мы берем символ на второй позиции и смотрим налево (на первый барабан, индекс 0). Если символ слева не совпадает и при этом не является Вайлдом — линия обрывается сразу же (`continue` / `break`). Если символ совпал — только тогда смотрим направо.

```
// Концепт ультра-быстрой проверки линии из 3 символов (позиции 0, 1, 2)
if ($flatReels[1] !== $flatReels[0]) {
    continue; // Линия оборвана, первый и второй символы не сошлись
}
if ($flatReels[1] !== $flatReels[2]) {
    continue; // Линия оборвана, второй и третий не сошлись
}
// Если дошли сюда — у нас зафиксирован выигрыш из 3 символов
```

В реальных проектах один Вайлд может не заменять Вайлд второй версии, Скаттер или какой-нибудь специфический бонусный символ. Сложная логика превращается в мгновенный поиск по ключу с помощью статических карт — **Lookup Tables (LUT)**:

```
$wildMap = [
    'WILD_V1' => ['A' => true, 'B' => true, 'C' => true],
    'WILD_V2' => ['D' => true, 'E' => true],
];
```

Память: Выдели один раз и перезаписывай

Динамическое создание массивов в цикле = аллокация + уборка. К примеру, пустое игровое поле создается один раз при инициализации сервиса и заполняется null или пустыми символами. В

процессе эмуляции мы не генерируем новый массив результатов, а просто перезаписываем значения по существующим индексам плоской структуры.

Микро-оптимизации на уровне ядра PHP

Что делает код медленным	Как нужно писать в Hot Path	Почему это работает
<code>count(\$array)</code>	<code>\count(\$array)</code>	Вызов из глобального пространства (<code>\</code>) исключает этап поиска функции в текущем namespace.
<code>array_merge()</code> , <code>array_flip()</code>	Ручной итерационный <code>for</code>	Встроенные функции выделяют новую память под массив. Простой <code>for</code> идеально компилируется JIT-ом.
Использование <code>float</code> (0.05)	Использование <code>int</code> (5)	Все расчеты переводятся в scaled x100 единицы (центы/копейки). Процессор считает <code>int</code> в разы быстрее и точнее.
<code>random_int()</code>	<code>\mt_rand()</code>	Криптографическая безопасность избыточна. На дистанции закон больших чисел сглаживает огрехи, но скорость возрастает кратно.

Критически важно: Никакого `xDebug` в среде тестирования. Он должен быть полностью отключен (*turn off*), так как его оверхед на логирование шагов убивает производительность на корню.

Разделение сред: Production vs Emulation

В Production режиме вам необходим честный, воспроизводимый и сертифицированный ГСЧ на базе шифрования (HMAC-SHA256). Но этот функционал — убийца скорости номер один. В режиме Emulation (Монте-Карло) эта логика полностью отключается и заменяется на инлайновый `\mt_rand`.

Также эмуляция требует нулевого I/O. Любой вывод логов, запись в файловую систему или даже банальный `echo` в консоль на каждом спине опускает скорость движка до уровня плитуса. Запись идет порциями для обновления прогресс-бара, а финальные результаты сохраняются строго в конце выполнения.

4. Результаты бенчмарков

После написания абсолютно нового ядра, апгрейда до PHP 8.5 и включения JIT для прогрева циклов, результаты для простой игры 3x3 на 7 символов с 3 выигрышными линиями зафиксировались на следующих отметках:

Metric	Value
Iterations	10 000 000
Iterations per Second	304 853.12
Total Execution time	00:00:32.802

Итог: 10 миллионов спинов обсчитываются всего за 32 секунды на одном ядре. Чистая скорость составила 304 000+ спинов в секунду. Это ускорение в 60 раз (6000%). Процесс верификации математических моделей и настройки игр сократился с недель до минут. За счет таких сокращений растет производительность всей команды и соответственно прибыль бизнеса.

5. Граница разумного: Темная магия оптимизации

Можно ли разогнать PHP еще сильнее? Да, но каждая последующая оптимизация наносит тяжелый удар по читаемости, поддержке и масштабируемости кода:

- **Битмаски:** Перевод расчетов линий и комбинаций на битовые маски может дать еще от 25% до 50% прироста скорости.
- **Тотальный процедурный код:** Полный отказ от классов и перевод всего движка в один монолитный файл на чистых массивах и функциях. Код превращается в нечитаемую кашу.
- **Глобальное хранилище состояния:** Создание единого хранилища памяти на самом верхнем уровне приложения. Сервисы общаются только через него, полностью исключая генерацию новых структур данных.
- **Precomputed Win Combo Maps:** Предварительный расчет всех возможных выигрышных комбинаций на этапе инициализации игры (например, массив `[A_A_A] => pay_multiplier`). Требует огромного количества памяти, но убирает логику расчетов из рантайма.
- **SplFixedArray:** Для наших плоских массивов, где ключами выступают простые целочисленные индексы, создает лишь лишний оверхед. Стандартный упакованный массив (packed array) в PHP 8.5 оптимизирован лучше.
- **Масштабирование через Node.js:** Использование Node.js-раннера, который параллелит PHP-процессы на доступные ядра процессора, полностью убирая оверхед на I/O.
- **Версионирование софта:** Разделение репозиторий (ядро отдельно, каждая игра отдельно) позволяет обновлять общие инструменты без риска сломать или изменить логику уже сертифицированных игр.

6. Миграция на Go (Golang) за 3 недели

В определенный момент я решил пойти еще дальше и полностью закрыть архитектурные недостатки, заложенные в самой природе PHP. Я перенес всю математическую логику движка на Go (Golang).

Самое интересное: я никогда в жизни до этого не писал на Go. Мой стек ограничивался PHP/JS и немного TypeScript. Перенос логики занял у меня порядка 2–3 недель неполного рабочего дня. Это стало возможным благодаря тому, что у меня уже была готовая, вылизанная архитектура, очищенная

от платформенного мусора, а ИИ использовался в качестве жесткого контекстного поисковика для решения тысяч узкоспециализированных вопросов синтаксиса и рантайма.

Главный вывод из сравнения PHP и Go

В PHP самой большой болью оставалась динамическая структура плоских массивов — малейшая ошибка типизации или индекса приводила к скрытым багам в расчетах. Go полностью решил эту проблему на уровне компилятора:

- **Строгая типизация как защита от дурака:** Компилятор Go жестко контролирует типы данных, предотвращая рантайм-хаос (например, передачу `uint8` вместо кастомного типа `SymbolID`). Код либо компилируется и гарантированно работает быстро, либо не компилируется вовсе.
- **Результат на Go:** Использование аналогичных принципов (Data Locality, Lookup Tables под байтовые ключи символов, полное предвыделение `capacity` слайсов и выполнение расчетов `strictly` на стеке процессора без аллокаций в куче через Escape Analysis) позволило мне выжать **миллионы спинов в секунду на одном единственном ядре!**

7. Заключение

Низкая скорость PHP под высокими нагрузками — это миф, созданный теми, кто не понимает PHP. Понимание того, как интерпретатор работает с памятью (`zval`), как JIT оптимизирует плоские циклы, и как процессор использует свой кэш, позволяет писать код с «механическим сочувствием» (Mechanical Sympathy) к железу.

Да, код теряет привычную для веб-разработчика «красоту», но он приобретает истинную инженерную элегантность и приносит бизнесу доход за счет увеличения производительности, экономии ресурсов и сокращения цикла разработки без потери качества. Полные скриншоты бенчмарков и логи профилирования доступны в репозитории проекта.